Curious Walkers on a Satellite Planet: The Design of the Bored Machines Agents

Bored Machines project · Technical report · November 2026

Abstract

We describe software agents that walk across satellite images of the Earth with no task and no reward from outside. Each agent starts with an untrained neural network and teaches it to redraw what it sees. At every step it scores the places it could go next by how novel they look and how much a little practice on them would teach it, picks one at random in favour of high scores, and trains on it. When nothing has looked interesting for a while it becomes "bored" and teleports. The paper builds this system up one idea at a time, states the formulas as they are implemented, explains why the design changed along the way, and summarises what the walks have shown so far, including where the agents fall short.

1 Introduction

An **agent** here is a program that repeats a simple cycle: look at its options, pick one, learn from what happens. Most agents in machine learning are given a goal through a **reward**, such as points in a game. Ours are not. Nothing tells them what is worth seeing. They have only an internal measure of interest, which is why this area of research is called **intrinsic motivation** or **artificial curiosity** [1], [2].

The setting is deliberately simple. The world is a grid of about 2.2 million satellite image tiles covering the planet. The agent stands on one tile and can move to a nearby one. It has no map, no labels and no pretrained vision. Everything it can perceive comes from one small neural network that it trains while walking, which is why we call its perception **self-grown**.

The agents are part of an art and research project, *Bored Machines*, whose walks are meant to become films and, later, the training material for image generators. This paper covers only the agents. It is written for readers who know computer science but not this field: each section adds one component, and Section 7 assembles them. Section 8 explains why the design looks as it does, and Section 11 reports where it stands. Appendix A collects the notation and terms.

All numbers come from real runs. Unless noted otherwise, the examples use one 5,000-step walk that started in Paris (the "Bored" character, seed 0, Section 9), and the settings are those of the current agent.

2 The world: tiles, neighbours and crops

The images come from Major TOM Core-S2L2A [3], a global collection of Sentinel-2 satellite imagery cut into patches on a grid of roughly 10 km. We keep one image per grid cell, the earliest one, after removing images with more than 10% missing data. This leaves 2,182,212 **tiles**. Each is reduced to 256 × 256 pixels of colour, about 42 m per pixel, so a tile is about 10.7 km across. There is one image per place, so the world does not change over time.

Neighbours. Two tiles are neighbours if their centres lie within 15 km. Distance on a sphere is given by the haversine formula, where φ is latitude, λ is longitude and $R = 6371$ km:

$$d = 2R \arcsin \sqrt{\sin^2\Big(\frac{\varphi_2 - \varphi_1}{2}\Big) + \cos\varphi_1 \cos\varphi_2 \sin^2\Big(\frac{\lambda_2 - \lambda_1}{2}\Big)} \tag{1}$$

On this grid a tile has about six neighbours, and a normal step covers about 11 km. Neighbour lookups use a ball tree, a spatial index that answers radius queries without comparing against every tile.

Candidates. The set of options at a step, written C, is all neighbours of the current tile. With probability $p_{\text{far}} = 0.02$, one tile drawn uniformly from the whole planet is added as well. We call this a **far offer**. The agent is free to ignore it, and usually does. In the example walk there were 6.5 candidates per step on average.

Crops. The network never sees a whole tile. Instead, the agent looks at $K = 16$ square **crops** of 64 pixels, about 2.7 km across, each reduced by averaging to 32×32 pixels (Figure 1). The crop size acts as the agent's scale of attention. For scoring, the positions of the 16 crops are fixed for each tile, derived from a random generator seeded with the tile's number, so a tile looks the same at every visit. For training, the agent draws 16 new random crops. Section 8 explains why these two differ.

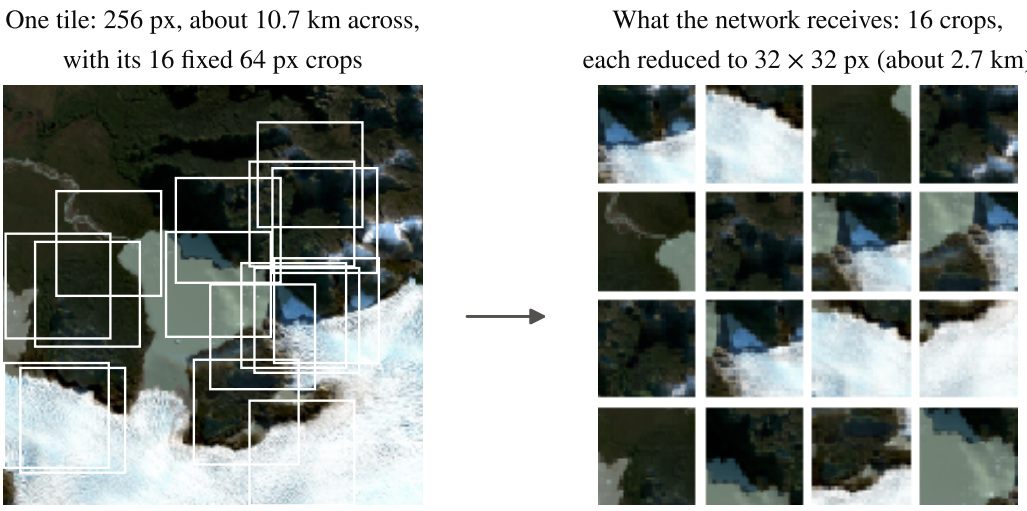


Figure 1: A tile and what the network gets from it. This glacier tile in Patagonia reappears in later figures.

3 Perception: an autoencoder that learns to draw

3.1 The autoencoder

An **autoencoder** [4] is a neural network trained to reproduce its input after passing it through a narrow middle layer. It has two halves: an **encoder** E turns a crop x into a short list of numbers z, called the **latent**, and a **decoder** D tries to rebuild the crop from z alone:

$$z = E_\theta(x), \qquad \hat{x} = D_\theta(z) \tag{2}$$

Here θ stands for all the network's adjustable weights. A crop holds $3 \times 32 \times 32 = 3072$ numbers and the latent holds 64, a reduction of 48 to 1. The network therefore cannot store crops. To redraw them well it must find regularities that terrain has in common, such as colour, edges and blobs. This is what we mean when we say the agent is **learning to draw**: $\hat{x}$ is its drawing of x, and z is its internal description.

Figure 2 shows the architecture. The encoder has four convolutional layers, each halving the image's width and height (32, 16, 8, 4, 2), followed by a linear layer to the latent. The decoder mirrors this. Convolutional layers slide small learned filters across an image, which suits pictures because a pattern means the same thing wherever it appears. The final sigmoid function keeps output pixels between 0 and 1. The network has 272,499 weights, starts from random values set by the agent's seed, and has no normalisation layers.

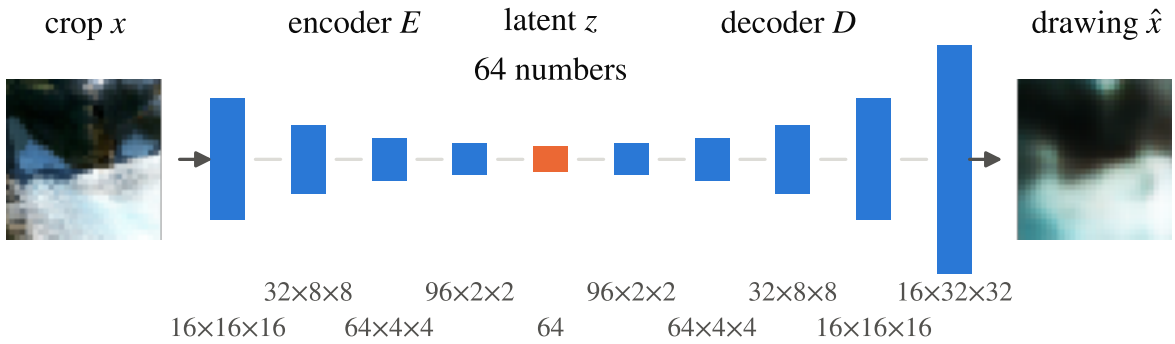


Figure 2: The autoencoder. Numbers give channels × height × width at each layer. The crop and drawing are real, from the glacier tile at step 5,000.

3.2 Measuring a drawing

The **reconstruction error** of a crop is the mean squared difference across all pixels and colour channels, with pixel values scaled to $[0, 1]$:

$$L(\theta; x) = \frac{1}{3 \cdot 32 \cdot 32} \| D_\theta(E_\theta(x)) - x \|^2 \tag{3}$$

For a sense of scale, an error of 0.002 is a typical difference of $\sqrt{0.002} \approx 0.045$, about 11 of 255 brightness levels. Because the error is squared, it grows with the contrast of the image: a flat patch of sea can hardly be wrong by much, while snow next to black rock can be wrong by a lot.

3.3 Learning: gradient descent and Adam

Training means nudging θ to lower the error. The gradient $\nabla_\theta L$, computed by backpropagation, tells us which direction raises the error for each weight, so we step the other way. We use **Adam** [5], the common variant that keeps a running average of the gradient, m, and of its square, v, and scales each weight's step by them. With $h_t = \nabla_\theta L$ at update t:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) h_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) h_t^2 \tag{4}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t)} + \varepsilon} \tag{5}$$

All operations are per weight. The defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$. The **learning rate** $\eta = 10^{-3}$ sets how quickly things become familiar. The factors $1 - \beta^t$ correct for the averages starting at zero.

3.4 Replay: a short memory

Neural networks trained only on the latest data tend to lose what they knew before, an effect known as **catastrophic forgetting** [6]. An agent that spends hundreds of steps on a glacier should not forget the sea. So the agent keeps a **replay buffer** [7]: a first-in, first-out store of the 2,048 most recent training crops, enough for the last 128 visits.

Each move triggers $u = 4$ Adam updates of Equation 4 and Equation 5 on one batch of 64 crops: the 16 fresh crops of the tile just chosen and 48 drawn at random from the buffer. A 5,000-step walk therefore makes only 20,000 updates in total. Figure 3 shows what that buys.

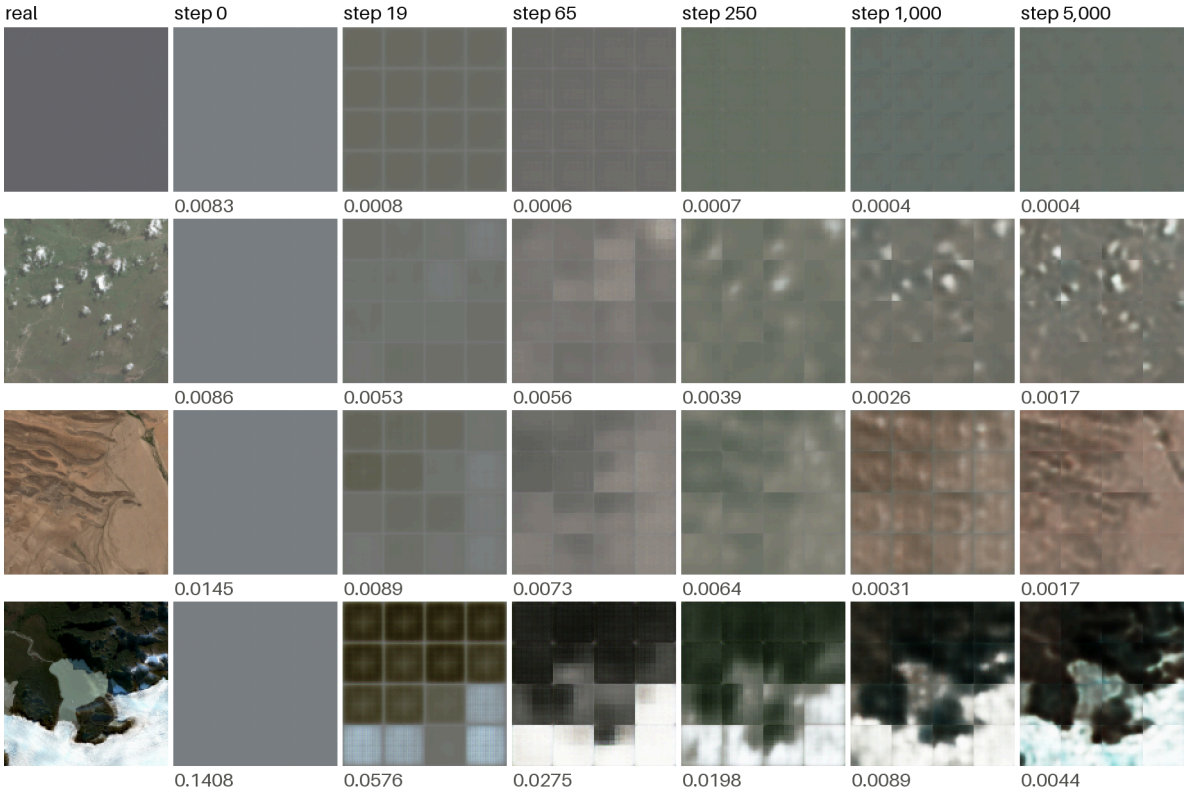


Figure 3: Learning to draw. Each row is a tile as the network receives it, then the drawing made by the same walker's network at increasing ages, with the error of that drawing. The number under each drawing is its error. The top two rows are held-out tiles the walk never visited. The bottom two were first visited at steps 493 and 3,800, so most of their drawings are of a tile the network has not yet trained on. Each tile is cut into a 4×4 grid of crops drawn separately, so the seams are the network's.

4 Three signals: what makes a tile interesting

Before moving, the agent looks at each candidate c through its 16 scoring crops $x_{c,1}, ..., x_{c,16}$ and computes three numbers. All use the network as it is at that moment.

4.1 Surprise: how badly can I draw it?

Surprise is the reconstruction error averaged over the candidate's crops:

$$e(c) = \frac{1}{K} \sum_{k=1}^{K} L\big(\theta; x_{c,k}\big) \tag{6}$$

This is the most obvious measure of curiosity, and it has a known flaw, the **noisy-TV problem** [8]: pure noise can never be predicted, so an agent seeking high error stays in front of static forever. We tested this by adding a few tiles of fresh random noise to an early regional world. An agent driven by surprise alone spent 77–96% of its steps on them when started nearby; the standard agent spent none. So surprise is logged, but in the current agents its weight is zero (Section 5).

4.2 Novelty: how unlike what I have seen is it?

Novelty compares a tile with memory [9]. A tile's description is the mean of its crops' latents, scaled to length 1 so that only its direction counts:

$$z(c) = \frac{1}{K} \sum_{k=1}^{K} E_\theta\big(x_{c,k}\big), \qquad \hat{z}(c) = \frac{z(c)}{\|z(c)\|} \tag{7}$$

The agent keeps an **archive** $\mathcal{A}$ holding one such latent for every visit, including repeat visits. Novelty is the mean distance to the $k = 10$ nearest entries (a k-nearest-neighbour distance):

$$n(c) = \frac{1}{k} \sum_{a \in \mathrm{kNN}_k(\hat{z}(c), \mathcal{A})} \|\hat{z}(c) - a\| \tag{8}$$

The archive needs upkeep, because the encoder keeps changing and old latents go out of date. The whole archive is recomputed with the current network every 100 steps, and a candidate's own entries are refreshed every time it is looked at. Memories are, in a sense, re-read by a changed mind.

Fixed crops and this refresh together give novelty an exact property. A tile's refreshed entries use the same crops and network as its new look, so they lie at distance zero. With v prior visits, v of the 10 nearest distances are zero, and a tile with 10 or more visits has novelty exactly 0. The logs confirm this: every candidate with at least 10 prior visits in the example walk has $n = 0$. Novelty therefore works as a built-in fatigue for individual tiles.

4.3 Progress: how much would a little practice teach me?

Learning progress [1], [2] is the idea that what is interesting is neither what is already known nor what is unlearnable, but what is being learned. We estimate it with a **probe**. Split the candidate's crops into halves A and B of eight crops each. On a temporary copy of the network, run $s = 3$ Adam steps on A, starting with fresh m and v, giving weights θ'. Then measure how the error on the **other** half changed:

$$g(c) = \frac{1}{|B|} \sum_{x \in B} L(\theta; x) - \frac{1}{|B|} \sum_{x \in B} L(\theta'; x) \tag{9}$$

Practising on A and testing on B is the key. Noise learned in one crop does not help with another, so noise should score low, which fixes the noisy-TV flaw. The copy is thrown away, so the agent only imagines the practice. In the code the copies are made functionally and batched with `torch.func.vmap`, so all candidates are probed in parallel.

In practice, progress is usually negative. In four walks we checked, $g < 0$ for 84–93% of all candidates, with a median around -0.001, comparable in size to the median error itself. So three steps of practice typically make the other crops *worse*. A likely reason is the fresh Adam state. At $t = 1$, Equation 4 and Equation 5 reduce to

$$\theta_1 = \theta_0 - \eta \cdot \frac{h_1}{|h_1| + \varepsilon} \approx \theta_0 - \eta \cdot \mathrm{sign}(h_1) \tag{10}$$

so every weight moves by the full η, however small its gradient. That is a sharp kick to a network otherwise being trained gently with accumulated averages. We have not tested this explanation separately. Either way, g is better read as "how little does a brief disturbance damage my drawing here?", and only its value relative to other candidates affects the choice.

5 From signals to a decision

5.1 Putting signals on one scale

The three signals have different units, and their typical sizes drift as the network learns. Each is therefore converted to a **z-score**: how many standard deviations it lies above its recent average. The average μ and variance σ^2 are **exponential moving averages** (EMAs), which forget old values gradually. At step t, with $\boldsymbol{x}$ the signal's values across the current candidates:

$$a_t = \max(\alpha, 1/t), \qquad \mu_t = (1 - a_t)\mu_{t-1} + a_t \,\mathrm{mean}(\boldsymbol{x}) \tag{11}$$

$$\sigma_t^2 = (1 - a_t)\sigma_{t-1}^2 + a_t \,\mathrm{mean}\big((\boldsymbol{x} - \mu_t)^2\big), \qquad \tilde{x}(c) = \frac{x(c) - \mu_t}{\sigma_t + 10^{-8}} \tag{12}$$

With $\alpha = 0.02$ the memory is about $1/\alpha = 50$ steps. For the first 50 steps, $a_t = 1/t$ makes this a plain average, so early scores are not inflated by a near-zero variance.

This has an important consequence: **interest is always relative to the last 50 or so steps.** A place cannot be interesting in absolute terms, and the average score cannot fade over a walk, because the yardstick moves with the agent.

5.2 Interest and choice

The **interest** of a candidate is a weighted sum of z-scores:

$$I(c) = w_n \tilde{n}(c) + w_g \tilde{g}(c) + w_e \tilde{e}(c), \qquad w_n = w_g = 1, \quad w_e = 0 \tag{13}$$

The agent does not simply take the best. It samples from a **softmax** distribution, which turns scores into probabilities:

$$P(c) = \frac{\exp(I(c)/T)}{\sum_{c' \in C} \exp(I(c')/T)} \tag{14}$$

The **temperature** $T = 0.5$ controls how firmly it prefers high scores. As T approaches 0 the agent always takes the top candidate, and as T grows the choice approaches uniform. A gap of 1 in interest means $e^{1/0.5} \approx 7.4$ times the probability. In practice the top candidate is chosen in 50–65% of steps, against about 15% if the agent chose blindly.

5.3 A real step



type	near	near	near	near	near	near	far
surprise e ($\times 10^{-3}$)	0.75	5.33	1.99	2.33	1.44	0.89	0.53
novelty n	0.35	0.16	0.57	0.45	0.15	0.31	0.15
progress g ($\times 10^{-3}$)	−0.36	−4.77	−0.28	−0.71	−7.01	−1.70	−3.56
$\tilde{n}$	0.40	−0.92	1.86	1.07	−0.92	0.11	−0.94
$\tilde{g}$	0.88	−0.99	0.92	0.73	−1.94	0.31	−0.48
interest I	1.28	−1.91	2.77	1.80	−2.87	0.42	−1.42
probability P	0.042	<0.001	0.831	0.119	<0.001	0.008	<0.001

Table 1: All candidates at step 424 of the example walk, from the log. The shaded tile was chosen. It has the highest novelty and the highest progress. Note that the tile with the highest surprise (second column) has low interest, and that the far offer (last column) is ignored.

Table 1 shows one decision in full. Interest is the sum of the two z-scores, and the probabilities follow from Equation 14: the chosen tile had $P = 0.83$.

5.4 Variants of the decision

Two switches change this stage. A **null walker** computes and logs everything but chooses uniformly, $P(c) = 1/|C|$. It is the control for "what does scoring actually do?". An optional **visit cost** subtracts $\beta \log(1 + \text{visits}(c))$ from $I(c)$ to discourage returning. It is off in all characters described here, because it would change the agent's temperament rather than correct its perception.

6 Boredom and teleports

After each move the agent asks whether even its best option was below its recent average, that is, whether $\max_c I(c) < 0$. **Boredom** B is an EMA of the answer:

$$B_t = 0.9\, B_{t-1} + 0.1 \cdot \mathbb{1}\left[\max_{c \in C_t} I(c) < 0\right] \tag{15}$$

where $\mathbb{1}[\cdot]$ is 1 if the condition holds and 0 otherwise. If B exceeds the agent's **patience** p, a **bored event** fires: the network is saved, a sheet of "daydreams" is decoded for later viewing, B resets to 0, and on the next step the agent **teleports**. A teleport offers a single candidate, drawn uniformly from the planet, so there is no choice. The agent scores it, moves there and trains on it. The landing step does not update B.

Starting from $B = 0$, a run of n dull steps gives $B = 1 - 0.9^n$, so the shortest run that can cause a teleport is

$$n = \left\lceil \frac{\log(1-p)}{\log 0.9} \right\rceil \tag{16}$$

This is 4 steps at $p = 0.3$, 9 at $p = 0.6$ and 16 at $p = 0.8$. Patience is therefore a very direct control on restlessness.

Figure 4 shows the first 125 steps of the example walk. There are six bored events in the first 66 steps, each following a short run of dull options; then a landing sticks, and the agent stays for hundreds of steps. We call this pattern **search by rejection**: landing, looking around, finding the surroundings dull, and leaving.

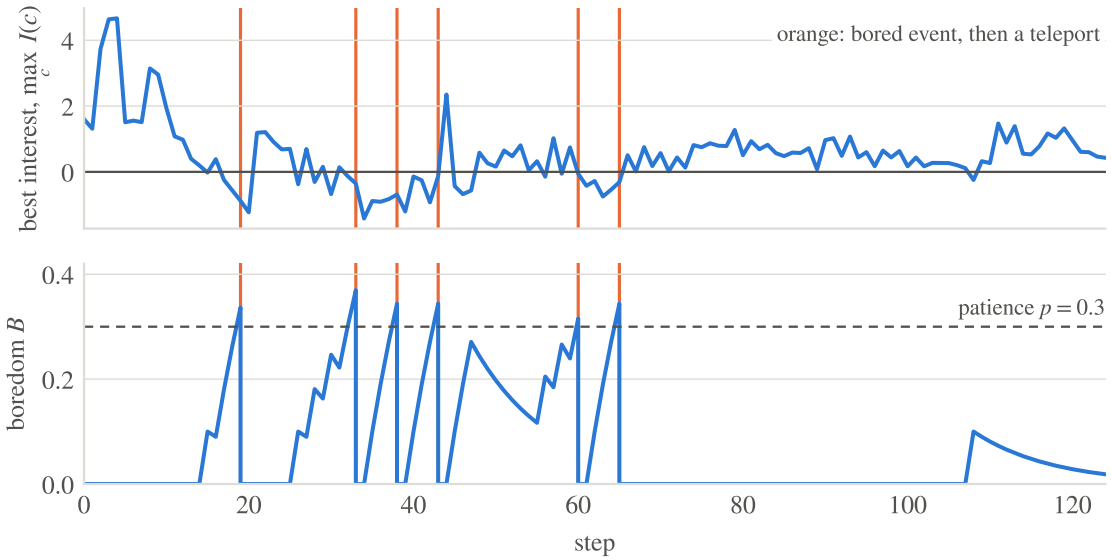


Figure 4: Boredom in the example walk. When the best interest stays below zero (top), B climbs to the patience threshold (bottom) and a teleport follows. The last landing, at step 66, begins a long stay.

An optional variant, **best-of-k landing**, draws k random tiles instead of one and lands on the most novel, giving the agent partial control over its teleports. Daydreams are decoded from random latents around the archive's most isolated entry. They are for viewing and do not steer the walker.

7 The complete agent

Figure 5 shows one step as a cycle, Algorithm 1 gives it in full and Table 2 lists the settings.

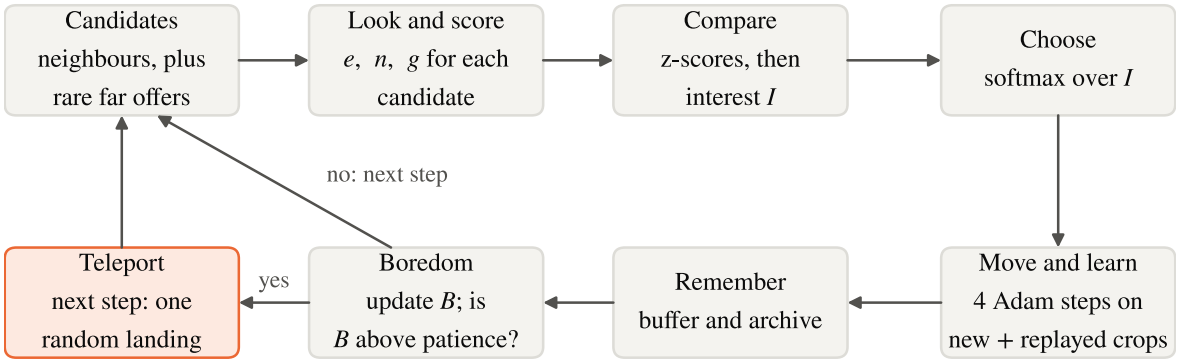


Figure 5: The cycle of one step.

Initialise: network θ from the seed; start tile t; empty archive $\mathcal{A}$ and buffer; $B = 0$.
For each step:

1. **Candidates.** If a teleport is pending, C = one random tile. Otherwise C = neighbours of t, plus one random far tile with probability p_{far}.
2. **Look.** For each $c \in C$, take its 16 fixed scoring crops.
3. **Score.** Compute $g(c)$ (Equation 9), $e(c)$ (Equation 6) and $z(c)$ (Equation 7). Refresh any archive entries of tiles in C, then compute $n(c)$ (Equation 8).
4. **Compare.** Update the running averages and form z-scores (Equation 12), then $I(c)$ (Equation 13).
5. **Choose.** Sample the next tile $t \sim P$ (Equation 14).
6. **Learn.** Draw 16 random crops of t and 48 replay crops; apply $u = 4$ Adam updates (Equation 4, Equation 5).
7. **Remember.** Add the 16 crops to the buffer; add $z(t)$, under the updated network, to $\mathcal{A}$.
8. **Boredom.** Unless this was a landing, update B (Equation 15). If $B > p$: save, daydream, set $B = 0$, mark a teleport as pending.
9. **Housekeeping.** Every 100 steps recompute all of $\mathcal{A}$; every 25 steps measure held-out error; every 250 save the network; log everything.

Algorithm 1: One walk.

Setting	Value	Meaning
K, crop size	16, 64 px	crops per look; scale of attention (2.7 km)
η	10^{-3}	learning rate: how quickly things become familiar
buffer	2,048 crops	replay memory, about 128 visits
u, batch	4, 16 + 48	Adam updates per move; new + replayed crops
radius	15 km	neighbour distance
p_{far}	0.02	chance of a far offer per step
k	10	neighbours in the novelty distance
s	3	Adam steps in the progress probe
α	0.02	z-score memory, about 50 steps
w_n, w_g, w_e	1, 1, 0	weights of novelty, progress, surprise
T	0.5	softmax temperature
p	0.6	patience (0.3 for Bored)

Table 2: The agent's settings. The values shown are those of the Reference character.

The cost is dominated by the probes, which run three training steps per candidate at every step. On a Mac CPU a step takes about a second, and the 5,000-step walk took 1.75 hours including downloading tiles on demand.

8 How the design got here

The current agent is the result of several rounds of correction. Three of the changes are worth explaining, because they illustrate how easily an apparent preference can be an artefact of measurement.

The first planet-scale walkers kept coming back. About 79% of steps landed on a tile already visited, 43% went straight back to the previous tile, and novelty actually *rose* with the number of visits, from 0.74 for a new tile to 1.21 for tiles visited more than 30 times. Analysis pointed to perception rather than taste, and three fixes followed:

- **Fixed scoring crops.** Originally each look drew new random crops. Measured with the final network, two looks at the same tile moved its latent almost as far as switching to a different tile (0.50 against 0.58, for tiles visited once). The tile could never become familiar: a noisy TV created by our own sampling. Crops for scoring are now fixed per tile. Training keeps random crops, so the network still sees all of a tile.
- **Fresh archive entries.** Stored latents went stale between the recomputations every 100 steps, which probably kept flat tiles looking novel. Candidates' own entries are now refreshed at every look (a hypothesis we adopted as a fix rather than tested on its own).
- **Normalised latents.** Latent length grew with visit count, from 4.4 for tiles visited once to 8.6 for tiles visited over 30 times, so plain distances rewarded "loud" tiles. Novelty now compares directions only (Equation 7).

With these fixes, Reference walks revisit a tile in about 58% of steps and go straight back in about 28%. This comparison is only indicative, as the walks differ in length and machine. More than half of those straight-back steps would happen anyway: a blind walker goes straight back 13–16% of the time purely from geometry.

The latest round added tools for controlled comparison: the null walker, best-of-k landings, a network-width factor, and **split random streams**. Previously one random generator served every decision, so changing one setting shifted every later random draw and the walks could not be compared. Now each kind of decision (far offers, landings, choices, initial weights, crops, replay, daydreams) has its own independent stream, derived from the seed by `numpy.random.SeedSequence`. Walks with the same seed now receive nearly identical far offers, and stay identical until a setting actually makes them differ. The example walk in this paper predates this change, which does not affect how the agent works.

9 The characters

A **character** is a named set of settings. Table 3 lists them. Each changes as little as possible from the Reference, so differences can be attributed, and each runs under several seeds, as the seed alone produces very different walks.

Character	Change from Reference	Idea
Reference	none	the standard agent
Bored	$p = 0.3$	loses patience after as few as 4 dull steps
Globalist	$p_{\text{far}} = 0.2$	ten times as many far offers
Localist	$p = 0.8$, $p_{\text{far}} = 0$	no far offers, slow to tire
Fast learner	$\eta = 3 \times 10^{-3}$	things become familiar sooner; unsafe, see below
Null	uniform choice	control: same rules, no use of scores
Landing variants	best-of-k, $k = 3, 8$	Reference and Bored with partial control of landings
Eye size	width $\times$ 0.5 or $\times$ 2	about 68k or 1.09M weights instead of 272k

Table 3: The characters. Round h0111 ran each of the Reference, Null, Bored and landing variants for 1,000 steps from Paris under four seeds. The eye-size runs used one seed each. The Globalist, Localist and Fast learner ran in an earlier round: two seeds, 1,000 steps from Milan.

The Fast learner shows the system can break. In one seed, at step 142, a few updates at the higher learning rate pushed the error on the chosen tile from 0.015 to 0.895. The decoder then produced black for every input, its sigmoid saturated, its gradient vanished, and the network never recovered.

10 Measuring what happens

Every step is logged, with the score of every candidate, so walks can be analysed afterwards without rerunning them. We rely on a few derived quantities:

- **Held-out error.** The mean reconstruction error, every 25 steps, on 200 fixed tiles drawn from the whole planet with fixed crops. It tracks how well the network draws the planet in general. These tiles are not used for training on purpose, but a walker could happen to visit one, so this is a fixed probe rather than a strict test set.
- **Place.** A stretch of a walk between two moves longer than 30 km.
- **Rejected landing.** A teleport whose place lasts at most 30 steps and ends in another teleport.
- **Appetite.** The fall in the logarithm of held-out error per 1,000 steps, over a 500-step window.

Figure 6 and Table 4 put held-out error in context with two simple comparisons computed on the same crops. One paints each crop in its own mean colour. The other is principal component analysis (PCA), the best linear compression, restricted to 64 numbers like the latent and fitted on the crops of the tiles the walk visited. PCA has an advantage, since it sees all its data at once and is fitted in one calculation, but it shows what 64 numbers can hold.

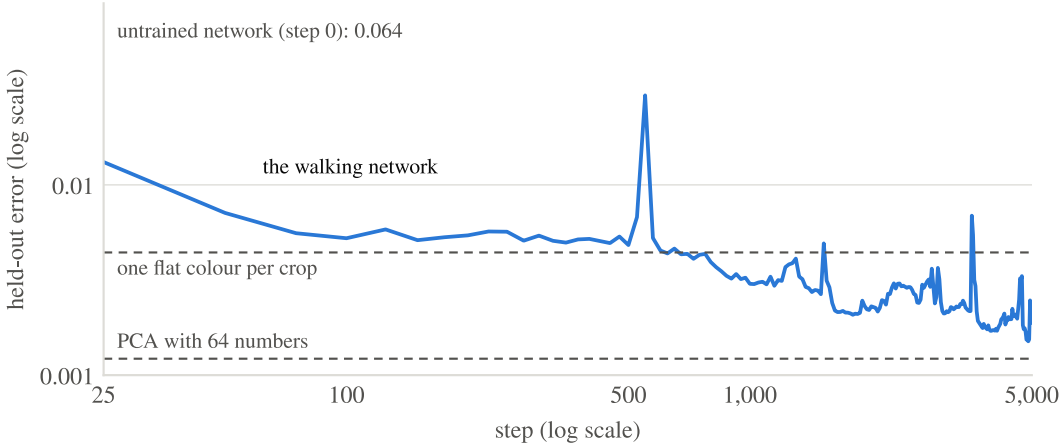


Figure 6: Held-out error of the example walk, with the two comparisons. The early fall is steep and the later one slow, and the curve is bumpy because training depends on where the walker happens to be.

Drawn by	Mean error	Median error	Crops beating flat colour
one flat colour per crop	0.0044	0.00053	–
PCA, 64 numbers	0.0012	0.00023	97%
network, step 65	0.0064	0.00212	7%
network, step 500	0.0048	0.00239	20%
network, step 1,000	0.0030	0.00095	40%
network, step 5,000	0.0019	0.00071	48%

Table 4: How good the drawing is. Errors on the 200 held-out tiles (3,200 crops). A few high-contrast crops dominate the mean, while the median reflects the many flat crops.

The picture this gives is modest. After 5,000 steps the network beats a flat colour on under half of held-out crops, probably because on flat terrain a small colour error costs more than the nearly absent detail is worth. Its mean error is about 1.5 times PCA's and its median about 3 times. Since 64 numbers can clearly hold more, we suspect the limit is the small amount of training rather than the network's size, though this has not been tested directly.

11 Where the agents stand

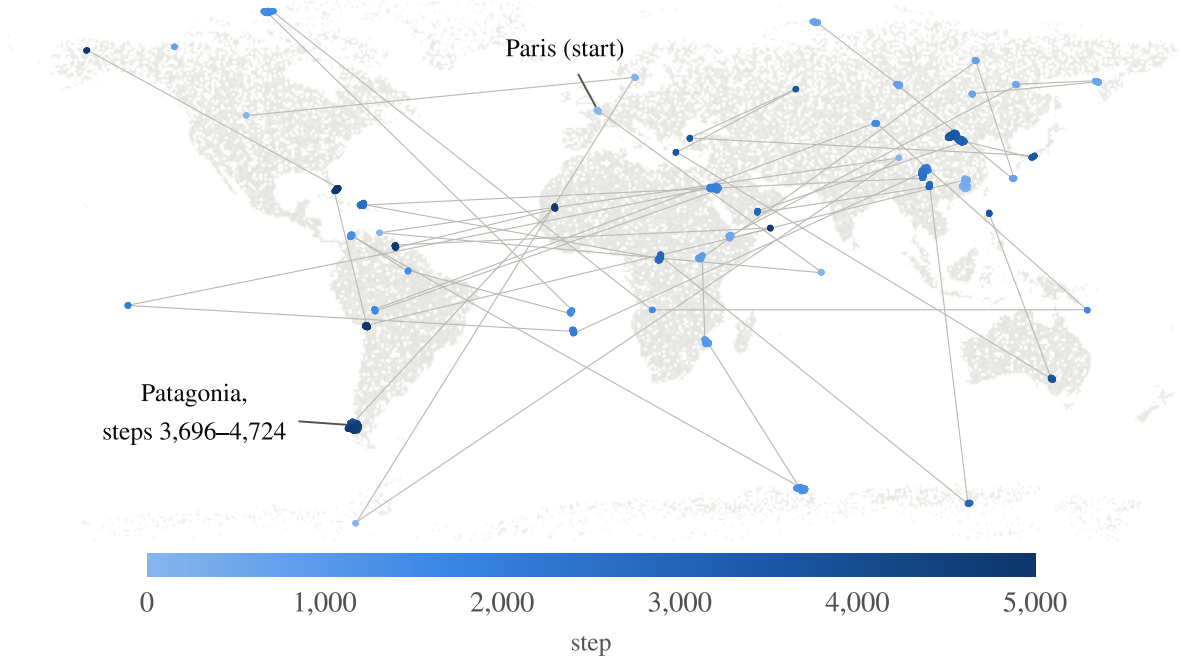


Figure 7: The example walk: 5,000 steps and 2,066 different tiles. Thin grey lines are its jumps: 35 bored teleports and 15 far offers taken. Colour shows time. Grey dots mark land.

The findings below are from one round of walks (four seeds per setting, 1,000 steps from Paris) and from twelve earlier walks. They are preliminary, and the sample sizes are small.

- **The agents settle on texture.** Long stays are on rougher terrain than short ones. The median ratio across walks is 1.5 for image gradient (higher in 10 of 12 walks) and 1.6 for contrast (12 of 12). Raw error is about equal (median ratio 1.04), so "interesting" is not simply "hard to draw". Because squared error grows with contrast, comparing raw errors across terrain is itself questionable.
- **Scoring sets the rhythm of a walk.** Removing it reduces places from 8.75 to 2.75 and teleports from 3.5 to 0.5 per walk, despite identical far offers. We expected the opposite. A long stay therefore does not by itself indicate attraction: the longest stays belong to the null walker, which chooses nothing.
- **Scoring does not visibly help learning.** Blind walkers see about as many tiles and end with held-out error at least as low (0.0038 against 0.0048, with overlapping ranges). In these agents curiosity shapes behaviour, and we have no evidence yet that it improves perception.
- **Search by rejection is robust.** Bored agents teleport 11–12 times per walk at every k. In the first 150 steps, 77–83% of landings are rejected, whatever the k. Choosing the most novel landing raises its novelty but does not prevent rejection, because rejection seems to depend on the surroundings of a landing rather than on the tile itself.
- **Learning has a fast phase and a slow one.** Mean held-out error falls tenfold by about step 65, largely by getting colour and brightness right (Figure 3). It then falls by a further factor of 2.5 between steps 500 and 5,000, which is when most of the visible drawing skill appears. Appetite, being a slope on a log scale, registers only the first phase.
- **Visits teach the terrain, not the tile.** One visit takes only 1–2% off a tile's error. Across 10 long stays in the example walk, visited tiles improved more than never-visited neighbours in 5 and less in 5. This is one walk with coarse checkpoints, but it fits an architecture too narrow to memorise.

Limitations. The strongest narrative claim, that a walk ends by settling somewhere, rests on one 5,000-step walk. Seed-to-seed spread is large and often exceeds the differences between settings.

Held-out error fluctuates by a factor of two or more over a few hundred steps. Several of our own predictions, written down before the runs, turned out wrong. Finally, words like "bored", "curious" and "interesting" name mechanisms in this paper. They make no claim about experience.

Next. The main open questions are whether settling is typical (more seeds at 5,000 steps), whether a blind walker with short patience becomes restless (separating the rule from the scorer), why rejection bursts gather early in a walk, and whether more training per step would improve drawing substantially.

12 Conclusion

The agent is small enough to state in about a dozen equations: an autoencoder trained online (Equation 3), novelty as distance in its latent space (Equation 8), progress as an imagined practice run (Equation 9), z-scores that keep everything relative to the recent past (Equation 12), a softmax choice (Equation 14) and a boredom counter (Equation 15). Each is simple, yet together they produce walks with recognisable structure: searches, long stays and departures. The more important lesson so far is how much care it takes to say what causes that structure, since some of what first looked like taste turned out to come from sampling, contrast or geometry.

Data. Major TOM Core-S2L2A (ESA Φ-lab), CC-BY-SA-4.0, containing modified Copernicus Sentinel-2 data.

References

[1] J. Schmidhuber, 'Formal Theory of Creativity, Fun, and Intrinsic Motivation (1990–2010)', *IEEE Transactions on Autonomous Mental Development*, vol. 2, no. 3, pp. 230–247, 2010.

[2] P.-Y. Oudeyer, F. Kaplan, and V. V. Hafner, 'Intrinsic Motivation Systems for Autonomous Mental Development', *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 2, pp. 265–286, 2007.

[3] A. Francis and M. Czerkawski, 'Major TOM: Expandable Datasets for Earth Observation', in *IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, 2024.

[4] G. E. Hinton and R. R. Salakhutdinov, 'Reducing the Dimensionality of Data with Neural Networks', *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[5] D. P. Kingma and J. Ba, 'Adam: A Method for Stochastic Optimization', in *International Conference on Learning Representations*, 2015.

[6] R. M. French, 'Catastrophic Forgetting in Connectionist Networks', *Trends in Cognitive Sciences*, vol. 3, no. 4, pp. 128–135, 1999.

[7] L.-J. Lin, 'Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching', *Machine Learning*, vol. 8, pp. 293–321, 1992.

[8] Y. Burda, H. Edwards, D. Pathak, A. Storkey, T. Darrell, and A. A. Efros, 'Large-Scale Study of Curiosity-Driven Learning', in *International Conference on Learning Representations*, 2019.

[9] J. Lehman and K. O. Stanley, 'Abandoning Objectives: Evolution Through the Search for Novelty Alone', *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.

Appendix A: Notation and terms

Term	Meaning
tile	one 256 px satellite image, about 10.7 km across
crop, x	a 64 px square from a tile, reduced to 32×32 px
$E_\theta, D_\theta, \theta$	encoder, decoder and their weights
latent, z	the 64 numbers that describe a crop; $z(c)$ is the mean over a tile's crops
drawing, $\hat{x}$	the decoder's reconstruction of a crop
reconstruction error, L	mean squared pixel difference between crop and drawing
held-out error	mean L on 200 fixed tiles from across the planet
candidate, c, C	a tile the agent could move to; the set of them at a step
far offer	a random tile from anywhere, added to C with probability p_{far}
surprise, e	error on a candidate before any practice
novelty, n	mean distance to the 10 nearest archive entries
progress, g	change in error on half the crops after three practice steps on the other half
archive, $\mathcal{A}$	one latent per visit; used for novelty
replay buffer	the last 2,048 training crops; used for training only
z-score, $\tilde{x}$	a signal measured against its recent mean and spread
EMA	exponential moving average: a running average that forgets gradually
interest, I	weighted sum of z-scores
softmax, T	turns scores into probabilities; temperature sets how firmly
boredom, B	running share of steps where even the best option scored below zero
patience, p	the value of B that triggers a teleport
teleport, landing	a forced move to a random tile; the tile it arrives at
place	a stretch of a walk between jumps longer than 30 km
character	a named set of settings

Appendix B: Where it is in the code

Component	Location
Autoencoder (Figure 2)	`agent/model.py`
Crops and fixed crop positions	`signals.patches`, `signals.fixed_offsets`
Surprise (Equation 6), latent (Equation 7), novelty (Equation 8)	`signals.recon_err`, `mean_latent`, `novelty`
Progress probe (Equation 9)	`signals.progress_probe`
Running z-scores (Equation 12)	`signals.RunningNorm`
Loop, choice, training, boredom (Algorithm 1)	`agent/walk.py`
Tile store and neighbours	`data/tile_store.py`
Character settings (Table 3)	`configs/h0111_*.yaml`, `configs/h2910_*.yaml`